Matrix Factory Twin: P2 Implementation Guide (OMNeT++ 6.4 + INET 4.7.0)

Prepared by Claude · Oct 12, 2026 · Target: all six backbone/radio cells operational by P2 (Nov 9, 2026)

How to read this guide

Every step has five parts: **What** to do, **Where**, **How**, **Why** (the purpose) and **Done when** (the exit test that tells you to move on). Tags: **[V]** means I verified the point by reading the INET 4.7.0 source; **[T]** means it is my best reading and must be tested on your first run. I have not built or run anything, so treat commands as starting points and trust your own error messages over this text. Companion files: `MatrixFactoryNetwork.ned` and `omnetpp.ini` from earlier in this chat.

Schedule at a glance

Week	Dates	Phases	Milestone
1	Oct 12-18	0, 1, 2	Decisions frozen, tools built, INET examples understood
2	Oct 19-25	3, 4, 5	Cell 1 runs, validated and measured; scheduled-wireless spike started
3	Oct 26-Nov 1	6, 7	Cells 3 and 5 working; scheduled-wireless approach decided
4	Nov 2-8	8, 9, 10	Six cells, pilot, hand-off package; Nov 9 = P2 due

The critical path is Phase 7 (scheduled wireless). INET has nothing ready-made for it, so start the spike in week 2 even though Cells 3 and 5 feel more urgent.

Phase 0: Freeze decisions before writing code (Day 1)

Step 0.1 Create the repository and the parameter sheet

- **What:** A git repository plus `docs/parameters.md`: one row per value that affects results (parameter, value, source, status = placeholder or confirmed).

- **Where:** Project root, e.g. `matrixfactory/`. Suggested layout: `simulations/` (NED + INI), `src/` (C++ extensions), `analysis/` (Python), `docs/`, `results/` (git-ignored).
- **How:** Copy every value from `omnetpp.ini` into the sheet: wired rate 100 Mbps; 802.11g at 54 Mbps; packet sizes (orders 600 B, state 1000 B, DDS 200 B, telemetry 1400 B) and intervals; PCP map (orders 6, DDS 4, state 2, telemetry 0); 1 ms gate cycle with its four windows; queue sizes (20 packets per class, 80 for the FIFO); run length 60 s; warm-up 5 s.
- **Why:** Every number is currently a placeholder. The P3 campaign and the lookup tables are only defensible if each value has a source, and one sheet keeps the six cells from drifting apart.
- **Done when:** Every value in the INI appears in the sheet, marked placeholder or confirmed.

Step 0.2 Ask the questions that change the model

- **What:** Send your professor and the twin maintainers: (1) What is the tick period T and the exact rule for the consumption instant? (2) What is the deadline per traffic class? (3) In what format and language will the NER engine read the lookup tables, and does it interpolate between grid points? (4) Is UDP acceptable instead of MQTT/TCP for VDA 5050 in P2? (5) What fleet sizes, station counts and link speeds does the real Matrix Factory use? Also ask for real message captures or sizes.
- **Where:** Email or the course channel; log answers in `docs/decisions.md`.
- **Why:** Questions 1-2 define your primary metric, question 3 decides what the simulator must record, and questions 4-5 set the load ranges.
- **Done when:** Each question has an answer, or (if none by Day 3) a logged assumption. Do not block on replies.

Step 0.3 Write the miss definition as formulas

- **What:** Choose one definition. Let g = generation time, a = arrival time at the consumer (infinite if lost), T = tick period, D = class deadline, L = minimum lead time.
  - **A, relative deadline:** miss if lost or a - g > D.
  - **B, tick grid:** consumption instant c = ceil((g + L) / T) x T; miss if lost or a > c.
- **Why:** Definition B depends on where g falls inside a tick, so a delay histogram alone cannot reproduce it: you need (g, a) per packet or an online check (Step 5.3). Choosing now avoids re-running the campaign later.
- **Done when:** The chosen formula is in `docs/decisions.md`.

Phase 1: Environment (Days 1-2)

Step 1.1 Install OMNeT++ 6.4.0

- **What:** A working OMNeT++ 6.4 with the IDE and command-line tools.
- **Where:** Linux is easiest (Ubuntu 24.04); on Windows use the MinGW shell (`mingwenv.cmd`) or WSL; macOS is supported.
- **How:** Download 6.4.0 from omnetpp.org and follow `doc/InstallGuide.pdf` (Linux: install prerequisites, then `source setenv`, `./configure`, `make -j$(nproc)`). INET's install notes recommend `opp_env` for a reproducible OMNeT++ + INET setup [V]; run `opp_env list` for exact package names.
- **Why:** INET 4.7.0 requires OMNeT++ 6.4 or later [V].
- **Done when:** `which omnetpp` works and a bundled sample (e.g. `samples/aloha`) runs.

Step 1.2 Build INET 4.7.0

- **What:** INET 4.7.0 in release and debug builds.
- **Where:** Next to OMNeT++, or inside your IDE workspace.
- **How:** `git clone --branch v4.7.0 https://github.com/inet-framework/inet.git`, then `cd inet`, `source setenv`, `pip install -r python/requirements.txt`, `make -j$(nproc)` and `make MODE=debug -j$(nproc)` [V, INSTALL.md]. `setenv` defines `INET_ROOT` [V]. If you unpack a tarball on Windows, use command-line `tar`, not the built-in extractor [V].
- **Why:** Release mode for campaigns; debug mode for crashes and assertions.
- **Done when:** `cd $INET_ROOT/showcases/tsn/gatescheduling/eager && inet` opens and runs.

Step 1.3 Python analysis environment

- **What:** A virtual environment with pandas, numpy, scipy and matplotlib, plus access to OMNeT++ results.
- **How:** `python3 -m venv .venv`, install the packages, then learn `opp_scavetool` (CSV export) and OMNeT++'s Python result API (check its docs for use outside the IDE).
- **Why:** The miss metric, confidence intervals and lookup tables are all computed offline.
- **Done when:** A script loads one `.sca` file from a sample run into a pandas DataFrame.

Step 1.4 Pin versions

- **What:** Record exactly what you run.

- **How:** In `README.md` write the OMNeT++ version, the INET tag and commit (`git -C $INET_ROOT rev-parse HEAD`), compiler and OS. Do not upgrade during P2 or P3.
- **Why:** Results from different versions are not comparable bit for bit: OMNeT++ 6.4 changed floating-point summation in statistics, and INET 4.7 changed how compound-queue statistics handle warm-up.
- **Done when:** README lists the versions.

Phase 2: Learn the pieces from INET's own examples (Days 2-4)

Step 2.1 Run the automatic gate-scheduling showcase

- **Where:** `$INET_ROOT/showcases/tsn/gatescheduling/eager` (config `Eager`).
- **How:** Run it in Qtenv, open a switch egress port, find `macLayer.queue.transmissionGate[0..1]`, watch the gate states.
- **Why:** It shows how INET identifies streams, tags a PCP and shapes traffic [V]: `hasOutgoingStreams`, a stream-identifier mapping by UDP port, an encoder mapping stream to PCP, and `Ieee8021qTimeAwareShaper` at the egress.
- **Done when:** You can point to each of those four pieces in the running model.

Step 2.2 Run the manual-gates showcase

- **Where:** `$INET_ROOT/showcases/tsn/trafficshaping/timeawareshaper` (configs `NoTrafficShaping`, `TimeAwareShaping`).
- **How:** Compare the two configs. A gate is written with `durations` (alternating open/closed lengths) and `initiallyOpen` [V]. Our TAS block also uses `offset`, which `PeriodicGate` treats as your position inside the cycle at time 0 [V, from its C++].
- **Why:** Our `Wired_TAS` block follows this style, so you must be able to read a gate schedule.
- **Done when:** You can predict on paper when each gate opens for a given `durations` / `offset`, and the gate-state graph confirms it.

Step 2.3 Run the wired plus wireless AP examples

- **Where:** `$INET_ROOT/examples/wireless/wiredandwirelesshostswithap` and `examples/wireless/lan80211`.
- **How:** Note `Ieee80211MgmtApSimplified` / `Ieee80211MgmtStaSimplified`, `mgmt.accessPointAddress`, the AP's `wlan[*].address` and `Ieee80211ScalarRadioMedium` [V].
- **Why:** This is the wireless half of our topology.

- **Done when:** You can explain how a frame travels from a wireless host through the AP onto the Ethernet link.

Step 2.4 Read six source files (about 2 hours)

- **What:** `src/inet/node/tsn/TsnSwitch.ned`, `src/inet/node/wireless/AccessPoint.ned`, `src/inet/linklayer/ieee8021q/Ieee8021qTimeAwareShaper.ned`, `src/inet/linklayer/ieee8021q/PcpTrafficClassClassifier.ned`, `src/inet/queueing/gate/PeriodicGate.ned`, and `src/inet/linklayer/ieee80211/mac/coordinationfunction/Dcf.cc` (`channelGranted`, `processUpperFrame`).
- **Why:** When something misbehaves, the answer lives here. For instance the AP has a plain Ethernet interface with no TAS [V], which is why the TSN switch sits behind it.
- **Done when:** `docs/inet-notes.md` has one paragraph per file.

Phase 3: Bring up the skeleton (Days 4-7)

Step 3.1 Create the project and run Cell1

- **Where:** `simulations/` in your project; copy `MatrixFactoryNetwork.ned` and `omnetpp.ini` there and mark it as a NED source folder (IDE: Project Properties > OMNeT++ > NED Source Folders). The NED file has no `package` line, so it must sit at the root of a NED folder.
- **How:** From `simulations/`: `opp_run -n $INET_ROOT/src:. -l $INET_ROOT/src/INET -c Cell1 -u Cmdenv omnetpp.ini` (use `-u Qtenv` for the GUI).
- **Why:** To surface every configuration error before adding features.
- **Done when:** It starts. Errors are expected on the first try (Step 3.2).

Step 3.2 Fix the likely first errors, one at a time

- **Module or parameter not found** for the stream identifier, encoder or learner [T]: open Qtenv, expand `tsnSwitch.bridging`, read the real submodule names and correct the INI path.
- **`destAddress` cannot be resolved** [T]: check the names (`fms`, `amr[3]`, `station[1]`) and what `parentIndex()` / `ancestorIndex()` return. Both functions exist in OMNeT++ 6.2 and 6.4 [V].
- **Association or management errors** [T]: check `**.mgmt.numChannels`, the AP address and `accessPointAddress`.

- **Gate or queue parameter rejected** [T]: `numTrafficClasses`, `transmissionGate[*]` and `queue[*]` exist only when `hasEgressTrafficShaping = true` [V].
- **Why:** The skeleton was written from reading source, not from running it; the first errors are the cheapest way to find which assumptions were wrong.
- **Done when:** Cell1 runs with one AMR and almost no telemetry (set `telInterval` very large).

Step 3.3 Smoke test with one AMR

- **How:** Set `nAmr=1`. In Qtenv or the `.sca` file, check that orders sent by the FMS equal orders received by the AMR (minus in-flight packets), and likewise for state reports, DDS and telemetry.
- **Why:** Proves each of the four flows crosses the whole path before you add load.
- **Done when:** Sent and received counts match, within in-flight packets, for all four classes.

Phase 4: Prove the path (Days 7-9)

Step 4.1 Hand-calculate one delay

- **How:** With no load, a wired flow's delay is the sum over links of (frame bits / link rate), plus per-hop processing (zero by default [T]). Frame size = payload + about 58 B of headers and preamble (UDP 8, IP 20, Q-tag 4, MAC 14, FCS 4, preamble 8) + 12 B inter-frame gap. A Q-tag may or may not be present on a given link, so check the frame length in Qtenv. For the 200 B DDS packet at 100 Mbps this is about 21.6 µs per hop, roughly 43 µs over two hops.
- **Why:** If the simulator disagrees with arithmetic, some parameter is not what you think.
- **Done when:** Simulated and calculated delays agree within a few percent.

Step 4.2 Verify tagging and priority

- **How:** Look at the headers of frames at three points: leaving the AP toward the switch, leaving the switch toward the AP, and leaving the switch toward the FMS. Use a pcap (`*.recordPcap = true`; check which links it captures [T]) or Qtenv's packet log. Confirm that (a) nothing tagged (0x8100) reaches the AP's wireless side, (b) each class gets the intended PCP, and (c) in Cell 3 the classes land in the right queues. With 4 traffic classes INET maps PCP 0, 2, 4, 6 to queues 0, 1, 2, 3 [V, classifier code].
- **Why:** Tagging happens in one place (the switch) and everything downstream depends on it.
- **Done when:** Per-queue counters in Cell 3 match the class mix.

Step 4.3 Check MAC learning

- **How:** Inspect the switch's MAC table in Qtenv: AMR MAC addresses must be learned on the AP port, and downlink orders must not be flooded to other ports.
- **Why:** `TsnSwitch` disables MAC learning when stream features are on [V]; the skeleton turns it back on [T] because AMR addresses cannot be configured statically behind the AP.
- **Done when:** No flooding after the first frames of each AMR.

Step 4.4 Check equal offered load and reproducibility

- **How:** Run Cells 1, 3, 5 with the same seed and compare the number of packets generated per source: it must be identical. Then run Cell 1 twice with the same seed: results must be identical.
- **Why:** Paired comparison between cells only works if the offered load is the same. Keep the default RNG manager: INET 4.7's `GroupedRngManager` assigns streams in module-creation order and I found no support for `rng-N` mappings in it.
- **Done when:** Both checks pass.

Phase 5: Measurement layer (Days 9-11)

Step 5.1 Decide what to record

- **What:** At sinks: `packetLifeTime` (end-to-end delay, vector and histogram), `packets`, `packetJitter`, `missingPacketIndices` [V, PassivePacketSinkBase]. At switch egress: `queueingTime` and drop counters of the shaper [V]. At the 802.11 MAC: `packetSentToPeerWithRetry`, `packetDropRetryLimitReached` [V, Dcf.ned] and the pending-queue queueing time [T].
- **Why:** These cover delay, jitter, loss, queue behavior and the cause of each miss, as your proposal requires.
- **Done when:** The list is in `docs/metrics.md`, with the exact statistic names confirmed in Qtenv.

Step 5.2 Attribute delay to a domain

- **How:** Split each end-to-end delay into (1) wireless: MAC queueing plus air time; (2) switch: queueing and gate-admission wait in the egress shaper; (3) the rest. Use the statistics above; if they are not enough, add a `packetLifeTime` statistic at the switch ingress from the AP port [T].
- **Why:** The study's core claim is which domain dominates; this attribution is its evidence and cannot be reconstructed after the runs.

- **Done when:** For one loaded run, the three parts sum to the end-to-end delay.

Step 5.3 Build the deadline check in two stages

- **Stage 1 (pilot scale):** Enable per-packet vectors (the `Debug` add-on). In Python compute g = receive time - lifetime, apply the Step 0.3 definition, and take loss = generated - received.
- **Stage 2 (campaign scale):** Vectors for hundreds of runs are too large, so write `DeadlineSink` in C++ (`src/`): a subclass of INET's `PassivePacketSink` that counts on-time, late and lost per class and keeps a delay histogram. Plug it in with `*.fms.app[0].sink.typename` ; the UDP app's `sink` is replaceable [V]. Finish it before the P3 campaign.
- **Why:** Histograms lose the tick phase that Definition B needs.
- **Done when:** Stage 1 reproduces a miss rate you checked by hand on a tiny run.

Step 5.4 Choose warm-up and run length

- **How:** For the heaviest configuration, plot a moving average of delay over time; set `warmup-period` where it flattens (placeholder: 5 s) and make `sim-time-limit` long enough for stable estimates.
- **Why:** Start-up transients bias every statistic.
- **Done when:** The chosen values and the plot are in `docs/` .

Phase 6: Wired mechanisms, Cells 1, 3, 5 (Days 11-15)

Step 6.1 Cell 3 (802.1Q priority)

- **How:** Run `Cell3` under heavy telemetry. Orders (PCP 6) should go to queue 3 and telemetry to queue 0; strict priority means telemetry only moves when higher classes are empty.
- **Why:** This is the middle step of the wired comparison. It uses the same shaper module as the TAS cell with gates held open, so the only difference between Cells 3 and 5 is the gating.
- **Done when:** Per-queue counters confirm the class-to-queue mapping.

Step 6.2 Cell 5 (802.1Qbv): check the window arithmetic

- **How:** Compute whether frames fit their windows. INET's gates apply an implicit guard band, so a frame that cannot finish before the window closes waits for the next cycle [V]. With the placeholder values (approximate, 100 Mbps, 12 B gap included):

Class	Frame on wire	Time	Window	Frames per window
Orders	670 B	53.6 µs	100 µs	1
State	1070 B	85.6 µs	200 µs	2
DDS	270 B	21.6 µs	200 µs	9
Telemetry	1470 B	117.6 µs	500 µs	4

- Consequences: telemetry is capped near 44.8 Mbps (4 frames x 1400 B per ms); at a 600 µs interval four stations offer about 74.7 Mbps, so in TAS cells the telemetry queue saturates while FIFO carries it. That is expected: what matters is that critical classes are unaffected. Orders get one slot per cycle, so they can wait up to 1 ms for their window; this comes from the placeholder GCL and may deserve a wider window.
- **Why:** Window sizes drive both throughput and the gate-admission delay you want to study.
- **Done when:** The gate-state vectors match the intended windows and the table matches your measurements.

Step 6.3 Run the divergence test

- **How:** Run Cells 1, 3, 5 with light fleet and heavy telemetry (interval 300-600 µs). Expect critical-class delay and jitter to be worst with FIFO, better with priority and bounded with TAS.
- **Why:** If the cells do not diverge, there is no wired bottleneck to study: link rate or load is wrong, which the pilot (Phase 9) must fix.
- **Done when:** The three cells show visibly different critical-class delay distributions.

Phase 7: Scheduled wireless, Cells 2, 4, 6 (start Day 8, decide by Day 18, finish by Day 24)

Step 7.1 Understand the gap

- **What:** INET has no scheduled-access 802.11. PCF and HCCA are stubs [V], unchanged in 4.7.0. You need a conflict-free regime that still pays the same air time as CSMA/CA (preamble, headers, ACKs, inter-frame spaces) and removes only collisions and random backoff.
- **Why:** If the scheduled regime also dropped protocol overhead, it would win trivially and the comparison would be meaningless.

Step 7.2 Spike S1: a scheduled coordination function

- **Hooks I verified [V]:** `Ieee80211Mac` declares `dcf` as `<default("Dcf")> like IDcf`, so `mac.dcf.typename` can be swapped; `Dcf::channelGranted` is virtual; the contention backoff is `intrand(cw + 1)`, so `cwMin = cwMax = 0` gives zero backoff slots.
- **Idea [T]:** a class `ScheduledDcf : public Dcf` with parameters `slotIndex`, `numSlots`, `slotDuration`. Override `channelGranted`: if now is outside this node's slot (or the remaining time cannot fit one frame exchange), call `channelAccess->releaseChannel(this)` and set a timer to request again at the next own-slot start; apply the same check where a frame sequence finishes and more frames wait. AMRs get slots 0..N-1 and the AP slot N for downlink. Slot length must cover the longest data frame + SIFS + ACK + DIFS, computed from your PHY.
- **Where:** `src/ScheduledDcf.ned`, `.h`, `.cc`. In the INI: `*.amr[*].wlan[0].mac.dcf.typename = "ScheduledDcf"`, `*.amr[*].wlan[0].mac.dcf.slotIndex = ancestorIndex(3)`, the same for the AP, and `**.wlan[*].mac.dcf.channelAccess.cwMin = 0` and `cwMax = 0` [T].
- **How to build:** Create an OMNeT++ project that references INET (IDE: New OMNeT++ Project, then add INET under Project References), or use `opp_makemake` with INET include and library flags (follow INET's Developer's Guide). Add `Radio_Sched` back to Cells 2, 4, 6 and remove the tripwire.
- **Why this route:** It reuses INET's framing, ACK and PHY, so air-time parity with the CSMA cells is automatic.
- **Done when:** A prototype passes Step 7.3 tests 1 and 2.

Step 7.3 Acceptance tests for the scheduled regime

1. Zero collisions: `packetSentToPeerWithRetry` and `packetDropRetryLimitReached` stay 0 with 40 saturated AMRs.
2. One AMR alone: frame-exchange duration equals CSMA/CA with `cw = 0`.
3. Throughput matches the slot capacity you computed.
4. Uplink wait for a slot is roughly uniform over the cycle.

- **Why:** These show the regime is conflict-free yet pays the same overhead.

Step 7.4 Fix the alignment between wireless slots and the gate cycle

- **How:** Make the wireless cycle an integer multiple of the 1 ms gate cycle (or the reverse) and fix the phase offset (P2: offset 0; P3 may sweep or randomize it). Record the choice in the parameter sheet.

- **Why:** This offset is what produces the wireless-to-wired gate-admission delay of Section 3.3 of your proposal.

Step 7.5 Decision rule and fallbacks

- **S2:** a custom MAC derived from INET's `MacProtocolBase`, using `LMac` (TDMA, built for sensor networks with its own framing [V]) only as a structural reference, not a drop-in.
- **S3:** model the AMR-to-AP hop as an abstract scheduled link (an Ethernet-like link with a TDMA gate per AMR, no 802.11 overhead) and state this limitation in the report.
- **Rule:** On Day 18, if S1 passes tests 1-2, continue. Otherwise try S2 for at most a week, else take S3.
- **Why:** Cells 2, 4, 6 depend on this and the schedule has no slack.

Phase 8: All six cells (Days 24-27)

Step 8.1 Fairness checklist

- **How:** Confirm the six cells share topology, traffic, seeds, the tagging pipeline, buffer budget (80 packets per port, i.e. 4 x 20), PHY rate and frame sizes. Only queue discipline and channel access may differ.
- **Why:** Any other difference is an artifact that will be mistaken for a result.
- **Done when:** Each item is ticked in `docs/fairness.md`.

Step 8.2 Run all six at three operating points

- **How:** Light, medium and heavy load, 3 seeds each. Tabulate deadline-miss rate, delay percentiles and loss per class.
- **Why:** A first look at whether the expected ordering and bottleneck migration appear.
- **Done when:** You can explain every surprising number, or have logged it as a bug.

Phase 9: Pilot (Days 26-28)

Step 9.1 Find the saturation points

- **How:** Raise the fleet size until the CSMA cell's miss rate climbs; raise telemetry until the FIFO switch port saturates. Note both knees.
- **Why:** The study is about the region between the knees, where the bottleneck moves.

Step 9.2 Estimate the campaign cost

- **How:** Runs = cells x fleet levels x load levels x replications; the skeleton's sweep is 6 x 4 x 3 x 10 = 720. Multiply by the measured wall time per run and divide by your cores.
- **Why:** It must fit the P3 window (Nov 9-23).

Step 9.3 Choose the number of replications

- **How:** Run 10 replications, compute the confidence-interval half-width of the miss rate, and use the Jain formula n >= (z x s / (r x mean))^2 for a target relative error r (e.g. 5%).
- **Why:** Separates real effects from simulation noise [proposal reference 12].

Step 9.4 Fix the P3 grid

- **How:** Choose load and fleet levels that bracket both knees; write them to `docs/p3-grid.md`.
- **Done when:** The grid and its estimated cost are written down.

Phase 10: P2 hand-off checklist

- All six cells run from one INI; Cells 2, 4, 6 use the working scheduled regime (or the documented S3).
- Validation notes: hand-calculated delay, tagging, learning, equal load, reproducibility, gate windows.
- Measurement layer: statistics list, attribution method, deadline check.
- Parameter sheet, decisions log, version pins, pilot results and the P3 grid.
- The repository state tagged `p2` in git.

Risks and fallbacks

- **Scheduled wireless does not work:** take S2, else S3 (Step 7.5).
- **Wired never queues:** raise telemetry, converge more traffic on the switch-to-FMS port, or lower the link rate; decide in the pilot.
- **Tagging or learning problems at the AP port:** tag at the wired sources instead and keep a single untagged path for the radio side; ask on the INET mailing list or GitHub issues with a minimal config.
- **Runs too slow:** shorten `sim-time-limit`, trim levels, and use `opp_runall` on several cores or a lab machine.
- **Version trouble:** INET 4.6.0 with OMNeT++ 6.2 uses the same code for every module in our model [V, by diff], so it is a safe fallback.

Command cheat sheet

- Build INET: `source setenv && make -j$(nproc)`; debug: `make MODE=debug -j$(nproc)`.
- Run one cell: `opp_run -n $INET_ROOT/src:. -l $INET_ROOT/src/INET -c Cell1 -u Cmdenv omnetpp.ini`.
- List the runs of a sweep: add `-q runs` to the same command with `-c Cell1_Sweep`.
- Export results: `opp_scavetool` (check `--help` for the CSV options).
- Re-activate the environment in each new terminal: `source <omnetpp>/setenv` then `source $INET_ROOT/setenv`.